

LAB_9

From Efficient Algorithms to Hard Problems

Session Objectives

By the end of this session, you will be able to:

1. Model social network problems as decision and optimization problems
2. Distinguish between efficient vs computationally hard problems
3. Analyze verification vs construction
4. Classify problems into P, NP, NP-Complete, NP-Hard
5. Evaluate scalability limits in practice

Core Exercises

Exercise 1: Influencer Coverage – Minimal User Set (Ref. Vertex Cover variant)

Objective: Select the smallest group of users such that every user is either selected, or directly connected (friend) to at least one selected user.

This is the minimum dominating set problem. You are trying to influence the entire network using as few users as possible.

Graph representation:

- Nodes: users
- Edges: friendship (undirected)
- Input: adjacency list or matrix

Requirements:

1. Implement `is_valid_coverage(selected_users, graph)`
 - Input: list of selected user IDs, graph representation
 - Output: True if every node is either selected or adjacent to a selected node
 - Complexity target: $O(N + E)$
2. Implement `find_minimum_coverage(graph)`
 - Finds the exact smallest dominating set
 - Must be correct for small graphs ($N \leq 20$ for brute force)
 - You may implement brute force over subsets, or branch-and-bound
 - Return: (size, list_of_selected_users)
3. Implement `find_fast_coverage()`
 - Greedy approximation: repeatedly pick the node that covers the most still-uncovered nodes
 - Return: (size, list_of_selected_users)
 - Compare its result with the exact solution for small graphs

Complexity Analysis Questions:

1. Verification vs construction
 - What is the time complexity of verifying a given candidate solution?

- Why is verification considered "easy" even if finding the solution is hard?
- Give a concrete small graph example where verification is trivial but finding the optimum requires checking many possibilities.
- 2. Growth of runtime
 - If you use brute force over all subsets, how many subsets for $N=10$, $N=20$, $N=50$?
 - Why is $N=100$ impossible with brute force?
 - How is the greedy approximation scale for $N=1,000,000$?
- 3. Optimality guarantee
 - Can the greedy algorithm always find the minimum? Prove or give a counterexample.
 - For large graphs (millions of nodes), what do real social networks do instead of searching for the exact minimum?

Exercise 2: Conflict-Free Labeling – User Categorization (Ref. Graph Coloring)

Objective: Assign labels (integers: 0, 1, 2, ...) to users such that:

- Connected users (friends) have different labels
- The total number of distinct labels used is minimized

This is the graph coloring problem. The minimum number of labels = chromatic number.

Requirements

1. Implement `is_valid_labeling(labeling, graph)`
 - Input: labeling array where `labeling[i]` = label of user i , graph
 - Output: True if for every edge (u,v) , `labeling[u] ≠ labeling[v]`
 - Complexity: $O(E)$
2. Implement `assign_labels(k, graph)`
 - Attempts to color the graph using at most k colors
 - Returns (success, labeling)
 - Implementation: greedy backtracking (try colors for each node, backtrack if stuck)
 - For small $N \leq 30$, this is feasible; for larger, stop after reasonable time
3. Implement `find_min_labels(graph)`
 - Finds the smallest k such that `assign_labels(k)` succeeds
 - Approach: try $k = 1, 2, 3, \dots$ until success
 - Return: (minimum_k, labeling)

Complexity Analysis Questions

1. When k is too small
 - What happens if you try to `assign_labels(1)` for a graph with at least one edge?
 - How can you quickly detect impossibility without trying all assignments?
 - Give an example graph where $k=2$ works but $k=1$ fails.
2. Effect of graph density
 - For a complete graph K_n , what is the minimum number of labels?
 - For an empty graph (no edges), what is the minimum number of labels?
 - Which type of graph is easier to color? Why?
3. Checking vs finding

- Is verifying a given labeling easier or harder than finding one? Give complexity.
- Why is `assign_labels(k)` potentially exponential in N ?
- What is the worst-case number of recursive calls for $N=30$ with $k=3$?

Exercise 3: Ad Campaign Optimization – Budget vs Reach (Ref. Knapsack variant)

Objective: Maximize total influence (reach) under a fixed budget. Each user i has:

- `cost[i]` (in dollars, integer)
- `influence[i]` (number of people reached, integer)

You choose a subset of users. Total cost $\leq$ budget. Maximize sum of influence.

Requirements

1. Implement `maximize_reach(budget, costs, influences)`
 - Exact solution using dynamic programming (0/1 knapsack)
 - Return: (max_influence, selected_users_list)
 - Complexity: $O(N \times \text{budget})$
 - Works for budget $\leq 10,000$ and $N \leq 500$
2. Implement `is_within_budget(selection, costs, budget)`
 - Simple helper: sum of costs of selected users $\leq$ budget
 - Complexity: $O(N)$
3. Implement `fast_alternative_strategy(budget, costs, influences)`
 - Greedy by influence/cost ratio (like fractional knapsack but without fractions)
 - Return: (influence, selected_users)
 - Compare with exact solution for small instances

Complexity Analysis Questions

1. Scaling with N
 - For DP: why is runtime $N \times \text{budget}$, not exponential in N ?
 - What happens if $N = 10,000$ and budget = 1,000,000? Is DP still feasible?
 - Why does this problem become harder if costs are real numbers instead of integers?
2. Optimality in practice
 - For $N = 100$, budget = 10,000, DP runs in ~1M operations (fast).
 - For $N = 1000$, budget = 1,000,000, DP runs in 1B operations (slower but possible in C++).
 - For $N = 1,000,000$, budget = $1e9$: is DP possible? Why not?
 - When would you accept the greedy approximation?
3. Exact vs fast comparison
 - Construct a small counterexample where greedy ratio fails to find the optimum.
 - What is the worst-case approximation factor of the ratio-based greedy?
 - In real ad platforms, why might they still use heuristics instead of exact DP?

Final Integration Questions

1. Classification

For each exercise, classify: P - NP - NP-Complete - NP-Hard

2. Decision vs Optimization

Transform each into: "Does a solution exist under constraint k?" Explain why this matters.

3. Scalability

What happens when: users = 1,000, or users = 1,000,000? Which problems fail first?

4. Engineering Perspective

- When is approximation acceptable?
- What would real platforms do?

5. Reflection

- Why do some problems resist efficient solutions? What common structure do NP-Complete problems share?
- If $P = NP$ were proven, what would change for these three social network problems?
- What does NP-hardness imply for AI systems trying to "solve" scheduling, routing, or recommendation optimally?

6. Edge Cases

- Empty graph
- Fully connected graph
- Budget = 0
- Very sparse vs. very dense graphs

General Instructions

Repository Setup

Create branch: LAB_09_HardProblems

File Structure:

Follow the same structure of LAB_08

Git Requirements:

Same as for LAB_08

Documentation & Notation

At the End of the Session	Before Friday Midnight of the Same Week
Note: 5 • Each Team (of number BB) Upload at the end of the session a very short PDF document and name it TBB_LAB9_AES.pdf, containing for each exercise: the algorithm in Pseudo-Code (according to the rules explained in lecture 1) – Brief answers of the questions, • You can implement your algorithms to make sure that they are working, • NO AI-generated content (will be checked for authenticity), • NO code snippets.	Note: 15 • You finish all the implementation of all exercises, • You finalize the previous report and give it a new name TBB_LAB9_BFM.pdf and upload it before Friday midnight of the same week, containing for each exercise: same as previous + more complexity analysis and reflection + Test set for edge cases of your Algorithms' implementation, • AI-generated content is not recommended, • A link to the Team's Repository in GitHub.